Baseline Retrieval vs Cross-Encoder Re-Ranking for RAG

GitHub repository: https://github.com/sai986/rag_assignment/

1. Document and Setup

For this assignment I used the **EU Artificial Intelligence Act** as the source document. I picked this document mainly because it is long enough, mostly text-based, and also has a lot of rule-like content. It has definitions, obligations, exceptions, prohibited practices, high-risk AI rules, transparency requirements, penalties, and annexes. So it felt like a good document to test where normal retrieval works and where it can miss more specific context.

The PDF has 144 pages. I extracted the text page by page and kept the page number as metadata. This was important because later, when a chunk is retrieved, I can still trace it back to the original page in the PDF.

Extraction Notes

Initially I tried `pypdf`, but the extracted text had spacing issues inside words. For example, in the PDF the first line is clean:

```
REGULATION (EU) 2024/1689 OF THE EUROPEAN PARLIAMENT AND OF THE COUNCIL
```

But `pypdf` extracted it like this:

```
REGUL A TION (EU) 2024/1689 OF THE EUR OPEAN P ARLIAMENT AND OF THE CO UNCIL
```

This is a problem for retrieval because words like `artificial`, `system`, `risk`, or `Article` can get split wrongly. That would affect embeddings and matching. So I switched to PyMuPDF, and the extracted text was much cleaner.

I also removed repeated header and footer text from pages. Things like journal metadata, page markers, and ELI footer lines were not useful for answering questions. They would only add noise to the chunks and embeddings. I kept legal citations because those are part of the actual document content.

2. Chunking Strategy

I used the full document. I did not remove the recital section because some questions can still be answered better when the background explanation is available. At the same time, I did not treat each page as one chunk. Page breaks are mostly a PDF layout thing, so I used page numbers as metadata only.

The chunking logic was:

1. Extract cleaned text for each page.
2. Join the pages into one continuous document stream.
3. Keep track of the page number for every word.
4. Prefer legal structure boundaries like `CHAPTER`, `SECTION`, `Article`, `ANNEX`, numbered clauses, and list items.
5. Build chunks around 600 words where possible.
6. Keep the maximum chunk size around 900 words.
7. Add 100-word overlap between chunks, including when the overlap crosses a page boundary.

The current chunking output is:

Metric	Value
Total words	90,406
Structural units detected	1,968
Chunks created	168
Minimum chunk size	398 words
Maximum chunk size	812 words
Mean chunk size	637.5 words
Chunks below 250 words	0
Chunks above 900 words	0

Metric	Value
Page-spanning chunks	143

I kept the chunks slightly large on purpose. Since this is a legal document, one rule often depends on conditions and exceptions nearby. Smaller chunks may retrieve only part of the rule and miss the qualification.

3. Architecture Comparison

The baseline system uses a bi-encoder. In this setup, the query and each chunk are embedded separately. The chunk embeddings are computed once and saved. At query time, only the query embedding is computed, and then cosine similarity is used to rank the chunks.

The re-ranking system adds a cross-encoder after this first retrieval step. A cross-encoder reads the query and candidate chunk together, so it can judge the match more directly. The tradeoff is that it is slower, because it has to run once for every query-chunk pair. Because of that, I only used it on the top 10 candidates from the bi-encoder, not on the whole document.

4. Retrieval Pipelines

Both pipelines use the same extracted text and the same chunks. The difference is only in what happens after the first dense retrieval step.

Pipeline 1: Baseline Retrieval

The baseline pipeline uses dense retrieval with `sentence-transformers/all-MiniLM-L6-v2`. I downloaded the model and loaded it from the local folder.

- Embedding model: `sentence-transformers/all-MiniLM-L6-v2`
- Embedding dimension: 384
- Similarity function: cosine similarity
- Number of candidates retrieved: top 10
- Number of chunks sent to the LLM: top 3
- Chunk embeddings file: `outputs/chunk_embeddings.npy`
- Baseline retrieval output: `outputs/baseline_retrieval_results.csv`

Pipeline 2: Advanced Re-Ranking

The re-ranking pipeline starts with the same top 10 candidates from dense retrieval. Then it scores each `(query, chunk)` pair using a cross-encoder and sorts the candidates again based on that score.

- Initial candidates: top 10 from baseline dense retrieval
- Cross-encoder model: `cross-encoder/ms-marco-MiniLM-L-6-v2`
- Model source: local folder `models/ms-marco-MiniLM-L-6-v2`
- Re-ranked chunks sent to the LLM: top 3
- Re-ranked output: `outputs/reranked_results.csv`
- Latency output: `outputs/latency_results.csv`
- Prompt output: `outputs/llm_prompts.csv`
- Prompt structure: same prompt is used for baseline and re-ranked answer generation
- Answer output: `outputs/llm_answers.csv`

In the first run over 8 candidate queries, the cross-encoder moved some useful chunks from outside the baseline top 3 into the final top 3. In a few cases, chunks that were originally at baseline rank 4, 5, 6, 7, or even 10 moved into the re-ranked top 3.

The answer-generation prompt template is:

```
Answer using only the numbered passages below.
If the passages do not contain the answer, say that the document does not provide enough information.
Cite every factual claim using passage ids like [S1], [S2].
```

```
Context:
[S1] ...
[S2] ...
[S3] ...

Question: ...
```

I used the same prompt template for both pipelines. Only the three retrieved passages changed.

The full prompts for the selected failure queries are saved in `outputs/llm_prompts.csv`, and the generated answers are saved in `outputs/llm_answers.csv`. In the report I am not pasting every full prompt and answer because that becomes too long. Instead I am focusing on the rank changes and answer-level differences.

5. Failure Query Analysis

Here I looked for cases where the baseline retrieval found relevant material somewhere in the top 10, but the most useful chunks were not always in the top 3. These are good cases to check whether re-ranking helps.

Candidate Rank Shifts

Query	Chunk	Baseline Rank	Re-ranked Rank	Page Range	Shift
What obligations apply to providers of general-purpose AI models with systemic risk?	C0070	10	3	41-42	+7
What criteria are used to designate general-purpose AI models with systemic risk?	C0119	10	3	86-87	+7
What obligations apply to providers of general-purpose AI models with systemic risk?	C0119	7	1	86-87	+6
What transparency obligations apply when users interact with an AI system?	C0014	7	2	7-8	+5
What penalties apply for non-compliance with prohibited AI practices?	C0015	6	2	8-8	+4
Which AI practices are prohibited because they manipulate a person's behaviour?	C0030	6	3	17-17	+3
When can law enforcement use real-time remote biometric identification in public spaces?	C0020	6	3	11-11	+3
What criteria are used to designate general-purpose AI models with systemic risk?	C0116	5	2	83-84	+3

The strongest examples were the general-purpose AI/systemic-risk questions. For those, relevant chunks moved from ranks 7 and 10 into the final top 3. That matters because only the top 3 chunks are sent to the LLM.

Failure Query 1

Question:

What obligations apply to providers of general-purpose AI models with systemic risk?

Why this query is difficult:

The document has multiple related sections around general-purpose AI models. Some chunks talk about definitions, some about systemic-risk classification, some about copyright/training summaries, and some about provider obligations. Dense retrieval can find generally related chunks, but it may not always put the exact obligation chunks at the top.

Rank shift:

Chunk	Baseline Rank	Re-ranked Rank	Page Range	Reason
C0119	7	1	86-87	Promoted to the top for the systemic-risk obligations query.
C0070	10	3	41-42	Moved into the final top 3 even though it was last in the baseline top 10.

Answer comparison:

Pipeline	Observation
Baseline context	Uses chunks C0052, C0050, C0053.

Pipeline	Observation
Re-ranked context	Uses chunks C0119, C0053, C0070.
Baseline answer	The answer covered risk mitigation, model evaluation, cybersecurity, incident reporting, and general systemic-risk duties. It was useful, but it mostly sounded like a summary of background and broad duties.
Re-ranked answer	The answer was more directly about provider obligations and compliance. It included risk assessment and mitigation, incident reporting, cybersecurity, codes of practice, harmonised standards, and alternative ways of showing compliance.

Short generated-answer comparison:

- Baseline answer: covered the main duties like risk mitigation, model evaluations, cybersecurity, and incident reporting.
- Re-ranked answer: kept those points, but added clearer compliance mechanisms such as codes of practice, harmonised standards, and alternative means of demonstrating compliance.

My observation: re-ranking helped here. It moved C0119 from baseline rank 7 to rank 1 and C0070 from baseline rank 10 to rank 3. Because of that, the final answer became more specific and less like a general background summary.

Failure Query 2

Question:

When can law enforcement use real-time remote biometric identification in public spaces?

Why this query is difficult:

This question depends on exceptions and conditions. The answer is not just one simple sentence. It involves strict necessity, authorisation, fundamental-rights assessment, national law, urgency handling, and the allowed objectives. Dense retrieval found biometric-related chunks, but not all of them were equally useful for this exact question.

Rank shift:

Chunk	Baseline Rank	Re-ranked Rank	Page Range	Reason
C0083	4	2	52-53	Contains operative authorisation and assessment requirements, so it was moved into top 3.
C0020	6	3	11-11	Contains Member State authorisation and national-law condition, so it was moved into top 3.

Answer comparison:

Pipeline	Observation
Baseline context	Uses chunks C0019, C0017, C0045.
Re-ranked context	Uses chunks C0019, C0083, C0020.
Baseline answer	The answer was broadly correct. It mentioned permitted situations, authorisation, urgency handling, scope limits, impact assessment, and national-law requirements. But one of the baseline chunks was more about post-remote biometric identification, not the real-time public-space use asked in the question.
Re-ranked answer	The answer stayed closer to real-time remote biometric identification in publicly accessible spaces. It included fundamental-rights assessment, EU database registration, prior authorisation, urgency exceptions, national-law permission, and notifications.

Short generated-answer comparison:

- Baseline answer: mostly correct, but mixed in some less central context about post-remote biometric identification.
- Re-ranked answer: more directly answered the real-time public-space use case and included the key conditions promoted by re-ranking.

My observation: re-ranking helped by promoting C0083 from baseline rank 4 to rank 2 and C0020 from baseline rank 6 to rank 3. This replaced less central baseline context with chunks that better matched the exact wording of the question.

6. Latency Profiling

Initial latency over 8 candidate queries:

Measurement	Baseline Retrieval Time	Re-ranking Time	Total Re-ranking Pipeline Time
Mean over 8 candidate queries	0.010 s	1.419 s	1.429 s
Minimum	0.009 s	1.326 s	1.336 s
Maximum	0.012 s	1.580 s	1.591 s

For the two selected failure queries:

Query	Baseline Retrieval Time	Re-ranking Time	Total Re-ranking Pipeline Time
GPAI systemic-risk obligations	0.012 s	1.624 s	1.637 s
Real-time biometric identification by law enforcement	0.013 s	1.514 s	1.527 s

The latency difference is clear. Baseline retrieval is almost instant for this document. Re-ranking adds around 1.5 seconds per query because the cross-encoder has to score query-chunk pairs one by one. For this assignment that is acceptable because I am only re-ranking 10 candidates.

The prompts for the two selected failure queries are exported to `outputs/llm_prompts.csv`. The generated answers are in `outputs/llm_answers.csv`. I have not stored any API keys or provider credentials in the notebook, report, or repository.

7. Scaling Question

We should not run a cross-encoder on the full collection directly. The reason is that a cross-encoder cannot fully precompute document embeddings like a bi-encoder. Its score depends on the query and chunk together, so every new query needs fresh scoring for every candidate chunk.

So the practical design is a two-stage pipeline:

1. Use the bi-encoder to search the full chunk collection quickly.
2. Take only a small candidate set, here top 10.
3. Use the cross-encoder only on those candidates to improve the final ordering.

In this experiment, dense retrieval over 168 chunks took about 0.010 seconds per query. Re-ranking only 10 candidates took about 1.419 seconds per query. This shows why the cross-encoder is useful as a second-stage model, but not as the first search step.

8. Conclusion

Overall, the two-stage retrieval pipeline gave better context for the difficult queries I tested. The baseline retriever was very fast and usually found relevant chunks somewhere in the top 10. But in some cases, the best chunks were not in the top 3, and those are the only chunks sent to the LLM.

The cross-encoder added extra latency, around 1.5 seconds per query for 10 candidates, but it improved the final context. For the systemic-risk obligations query and the law-enforcement biometric identification query, re-ranking moved more exact chunks into the top 3. That made the generated answers more specific and better grounded in the document.